MediMate 2.0

Smart Medicine Reminder System

PRODUCT REQUIREMENTS DOCUMENT

Complete Build Guide | React Native + ESP32 + Firebase

Project	MediMate 2.0 — Smart Medicine Reminder System
Document Type	Product Requirements Document (PRD) + Step-by-Step Build Guide
Version	v1.0 — March 2026
Institution	Sir Visvesvaraya Institute of Technology (SVIT), Nashik
Department	Electronics and Computer Engineering
Academic Year	AY 2025–26
Project Guide	Dr. Rana Mahajan
Team Members	Rohit Vijay Pawar [30] | Tanuja Changdev Ranade [35] | Shruti Kailas Saskar [47]
Tech Stack	React Native (Expo) | ESP32 DevKit C | Firebase | Arduino IDE | Twilio
Document Status	Active — Single Source of Truth for Design, Development & Testing

This document defines every screen, feature, API, wiring diagram, code pattern, and weekly milestone required to build, test, and demonstrate MediMate 2.0.

Table of Contents

1. Executive Summary 3
2. Problem Statement & Market Gap 3
3. Project Vision, Goals & Success Metrics 4
4. System Architecture 5
5. Complete Tech Stack & Tools 6
6. Feature Specifications — All Features 7
7. React Native App — Complete Step-by-Step Build 10
8. ESP32 Hardware — Complete Step-by-Step Build 18
9. Firebase Backend — Complete Setup Guide 22
10. Multilingual System (Marathi / Hindi / English) 25
11. Caregiver Alert System (WhatsApp via Twilio) 26
12. Database Schema — Complete Reference 27
13. API & Data Flow Specification 28
14. Week-by-Week Development Roadmap (12 Weeks) 29
15. Testing Plan — Complete Checklist 31
16. Non-Functional Requirements 33
17. Risk Register & Mitigations 33
18. Demo Day Script & Checklist 34
19. Future Scope & Enhancements 35
20. Appendix: Common Errors & Fixes 36

1. Executive Summary

MediMate 2.0 is a full-stack, IoT-enabled smart medicine reminder system that solves one of healthcare's most persistent problems — medication non-adherence. It combines three tightly integrated layers into one seamless experience:

Layer	Technology	What It Does
Mobile App	React Native (Expo)	User adds medicines with photos, schedules times, selects language, views today's doses, marks taken, sees history
Cloud Backend	Firebase (RTDB + Storage + FCM)	Stores all data in real time, syncs to ESP32, fires push notifications, serves tablet photos
Hardware Unit	ESP32 + Speaker + RTC	Plays voice announcement through speaker at exact scheduled time — even without phone nearby

When reminder time arrives, three things happen simultaneously: the ESP32 speaks the medicine name through a physical speaker in the user's language, the phone receives a push notification, and tapping it opens a full-screen view with the tablet's photo and auto-plays a voice reminder. If the dose is not confirmed within 30 minutes, a WhatsApp message is sent to the caregiver. The system costs under ₹900 to build, runs on free-tier cloud services, and is designed specifically for elderly and chronically ill patients.

2. Problem Statement & Market Gap

2.1 The Core Problem

Medication non-adherence — patients not taking medicines on time or skipping doses — is responsible for approximately 125,000 deaths annually in the US alone and costs healthcare systems billions. In India, the problem is amplified by:

- Large elderly population managing multiple chronic conditions (diabetes, hypertension, thyroid, arthritis) simultaneously
- Complex schedules — some patients take 6–8 medicines at different times with food/without food conditions
- Language barrier — most reminder apps are English-only; elderly users in Maharashtra need Marathi
- Household dynamics — elderly users often live alone or caregivers are at work during the day
- Phone dependency — alarms are useless when phone is on silent, in another room, or battery dead

2.2 Why Existing Solutions Fail

Existing Solution	Critical Failure	MediMate Solution
Phone alarm	Silent on vibrate; no medicine context; no photo; no caregiver alert	Physical speaker + multi-channel alert + photo identification
Paper medication chart	No alert; easily forgotten; caregiver cannot monitor remotely	Real-time Firebase sync; caregiver WhatsApp alert; history log
Basic reminder apps (e.g. Medisafe)	No hardware backup; English only; no ESP32 integration	Hardware speaker + Marathi/Hindi/English voice + IoT sync
Hospital dispensing systems	Expensive (₹50,000+); designed for wards not homes	Under ₹900 hardware; free cloud tier; home-friendly
Smart pill boxes (imported)	No app; no voice; no photo ID; costs ₹5,000–₹15,000	Full app + voice + photo + caregiver alert under ₹900

2.3 Target Users

User Type	Description	Key Need
Primary Patient	Elderly (60+), multi-medicine, limited tech literacy	Marathi/Hindi voice + physical speaker + photo ID
Chronic Patient	35–60, diabetes/hypertension, busy lifestyle	Phone notification + history tracking for doctor visits
Caregiver	Family member, may not be home during day	WhatsApp alert if dose missed; remote adherence view
Healthcare Student	Project demonstration context	Technical depth + working prototype + demo-ready system

3. Project Vision, Goals & Success Metrics

3.1 Vision Statement

A smart, multilingual, IoT-backed medicine reminder that works even when the phone is silent, in another room, or in the hands of someone who cannot read — delivering zero-missed-dose reliability to elderly and chronically ill patients across India.

3.2 Core Goals

Goal ID	Goal	How MediMate Achieves It
G-01	Zero missed doses through redundant alerts	Speaker + push notification + in-app voice fire simultaneously
G-02	Photo-based medicine identification	User photographs tablet; shown full-screen at reminder time
G-03	Full Marathi, Hindi, English support	Voice, UI, and speaker audio all support 3 languages
G-04	Caregiver visibility into adherence	WhatsApp alert after 30 min; history screen accessible
G-05	Expiry prevention	Countdown badge + 7-day early warning notification
G-06	Offline hardware resilience	ESP32 caches schedule in SPIFFS flash — alerts fire without internet
G-07	Affordable, accessible build	Hardware under ₹900; app uses Firebase free tier indefinitely
G-08	Elderly-friendly UX	Large fonts (min 18pt), large buttons (min 48px), high contrast

3.3 Success Metrics (Demo Day Criteria)

Metric	Target	How to Measure
Reminder accuracy	Alert fires within 60 seconds of scheduled time	Set reminder 2 min ahead; time with stopwatch at demo
Multilingual voice	All 3 languages speak correctly	Test Marathi, Hindi, English live at demo
App → ESP32 sync latency	New medicine visible to ESP32 within 60 seconds	Watch Serial Monitor while saving in app
Caregiver alert	WhatsApp received within 35 minutes of missed dose	Test with demo phone; check WhatsApp
Offline operation	ESP32 fires alert with WiFi disabled	Disconnect router; confirm speaker still announces
Photo display	Tablet photo shows on ReminderScreen	Visible in demo walkthrough

Metric	Target	How to Measure
Expiry badge	Red badge shows for medicine expiring in 7 days	Add medicine with expiry = today + 5 days

4. System Architecture

4.1 Component Architecture

Layer	Component	Technology	Responsibility
Presentation	Mobile App UI	React Native + Expo SDK 51	All user-facing screens; form inputs; photo capture; history charts
Presentation	Push Notifications	Expo Notifications + FCM	Scheduled local alerts at medicine time; foreground/background/killed app
Presentation	In-App Voice	expo-speech (TTS)	Speaks medicine name in mr-IN / hi-IN / en-IN at ReminderScreen open
Logic	Navigation	@react-navigation/stack	Stack-based screen routing; deep link from notification to ReminderScreen
Logic	Local Storage	AsyncStorage	Persists user session, language preference, offline schedule cache
Logic	Notification Scheduler	expo-notifications	Schedules repeating daily notifications per medicine per time slot
Data	Realtime Database	Firebase RTDB	Medicine schedules; taken_today flags; user profiles; history logs
Data	File Storage	Firebase Storage	Tablet photos (JPEG, max 500 KB each)
Data	Authentication	Firebase Auth (Email/Pass)	User login; session management; UID-scoped data access
Alerts	Caregiver WhatsApp	Twilio WhatsApp API	30-min missed dose alert to registered caregiver number
Automation	Midnight Reset	Firebase Cloud Functions	Resets taken_today: false daily at 00:00; logs history entry
Hardware	Microcontroller	ESP32 DevKit C	WiFi; HTTP REST calls to Firebase; RTC comparison; alert triggering
Hardware	Real-Time Clock	DS3231 (I2C)	Accurate offline timekeeping; holds time through power cuts
Hardware	Audio Output	MAX98357A I2S + 3W Speaker	Digital-to-analog; amplified voice from pre-generated MP3 on SD card
Hardware	Alert Backup	Active Buzzer + LED	3-cycle beep + blink if speaker fails or audio file missing
Hardware	Audio Storage	MicroSD card + SPI module	Stores all pre-generated MP3 files (one per medicine per language)

4.2 Complete Data Flow — Step by Step

1. User opens React Native app and navigates to Add Medicine screen
2. User fills: medicine name, dose, reminder times (multiple allowed), expiry date, tablet photo, preferred language (mr/hi/en), active toggle

3. On Save tap: app compresses photo → uploads to Firebase Storage → gets download URL
4. App writes complete medicine object to Firebase RTDB at path: /schedules/{uid}/{medicineId}
5. App calls scheduleNotification() for each time slot — creates repeating daily local notification
6. ESP32 polls Firebase RTDB via HTTP GET every 60 seconds — fetches /schedules/{uid}.json
7. ESP32 saves latest successful fetch to SPIFFS flash memory (offline cache)
8. DS3231 RTC tracks real time continuously — ESP32 reads it every loop iteration
9. When RTC hour:minute matches a medicine's scheduled time → triggerAlert() fires
10. triggerAlert(): blinks LED 3 times + beeps buzzer 3 times + plays {medicine}_{lang}.mp3 from SD card via I2S speaker
11. Simultaneously: Expo Notifications fires local push notification on user's phone
12. User taps notification → app opens ReminderScreen with tablet photo (full screen) + auto-speaks via expo-speech
13. User taps "Mark as Taken" → app sets taken_today: true in Firebase RTDB + logs history entry + cancels caregiver timer
14. If taken_today remains false after 30 minutes → caregiver alert function calls Twilio API → WhatsApp message sent to caregiver
15. At 00:00 midnight → Firebase Cloud Function runs → resets all taken_today: false → writes yesterday's taken/missed to /history/{uid}/{date}

5. Complete Tech Stack & Tools

5.1 Mobile App (React Native)

Package / Tool	Version	Purpose	Install Command
React Native	via Expo SDK 51	Core mobile framework	— (included in Expo)
Expo	SDK 51	Dev toolchain; camera; notifications; speech	npx create-expo-app MediMate --template blank
@react-navigation/native	6.x	Navigation container	npx expo install @react-navigation/native
@react-navigation/stack	6.x	Stack (push/pop) navigation	npx expo install @react-navigation/stack
react-native-screens	~3.31	Native screen performance (required by nav)	npx expo install react-native-screens
react-native-safe-area-context	~4.10	Safe area insets for notches	npx expo install react-native-safe-area-context
firebase	10.x	All Firebase SDKs (RTDB, Auth, Storage)	npx expo install firebase
expo-notifications	~0.28	Local push notifications; scheduling	npx expo install expo-notifications
expo-speech	~0.11	Text-to-speech in mr-IN / hi-IN / en-IN	npx expo install expo-speech
expo-image-picker	~15.0	Camera + gallery photo selection	npx expo install expo-image-picker
expo-device	~6.0	Physical device detection (for notifications)	npx expo install expo-device
expo-av	~14.0	Audio playback (backup for TTS)	npx expo install expo-av
expo-print	~13.0	PDF export of adherence report	npx expo install expo-print
@react-native-async-storage/async-storage	~1.23	Persist session; offline schedule cache	npx expo install @react-native-async-storage/async-storage
react-native-localize	~3.2	Detect device locale	npx expo install react-native-localize
i18n-js	~4.4	Translation string management (en/hi/mr)	npm install i18n-js
react-native-chart-kit	~6.12	Weekly adherence bar chart in History screen	npm install react-native-chart-kit
react-native-svg	~15.4	Required peer dep for chart-kit	npx expo install react-native-svg

Package / Tool	Version	Purpose	Install Command
date-fns	~3.6	Date formatting and manipulation	npm install date-fns
expo-status-bar	~1.12	Status bar styling	— (included in Expo)

5.2 Hardware & Firmware

Component	Model / Tool	Purpose
Microcontroller IDE	Arduino IDE 2.x	Write, compile, upload C++ firmware to ESP32
Board Package	esp32 by Espressif v2.0.x	ESP32 board definitions and core libraries
RTC Library	RTClib by Adafruit	Read/write DS3231 real-time clock over I2C
JSON Parser	ArduinoJson by Benoit Blanchon	Parse Firebase REST JSON responses
I2S Audio	ESP32-audioI2S by schreibfaul1	Play MP3 files from SD card via MAX98357A
SD Card	SD by Arduino (built-in)	Read MP3 files from microSD card
HTTP Client	HTTPClient (built-in ESP32)	HTTP GET to Firebase REST API
SPIFFS	SPIFFS (built-in ESP32)	Store offline schedule cache in ESP32 flash
Audio Generation	Python gTTS library	Generate MP3 reminder files on PC before build
Python Environment	Python 3.10+ on your PC	Run gTTS script; install: pip install gtts

5.3 Cloud & Backend

Service	Plan	Limits on Free Tier	Used For
Firebase Realtime Database	Spark (Free)	1 GB storage, 10 GB/month transfer	Medicine schedules, user profiles, history
Firebase Storage	Spark (Free)	5 GB storage, 1 GB/day download	Tablet photos (avg 200 KB each = 25,000 photos)
Firebase Authentication	Spark (Free)	10,000 sign-ins/month	User login/registration
Firebase Cloud Functions	Spark (Free)	125,000 invocations/month	Midnight reset, caregiver alert trigger
Firebase Cloud Messaging	Free always	Unlimited push notifications	Push notifications (backup to local)
Twilio WhatsApp API	Trial	₹750 trial credit; $0.005/msg after	Caregiver missed-dose WhatsApp alert

6. Feature Specifications — All Features

6.1 Must-Have Features (Version 1.0 — Core Demo)

ID	Feature Name	Description	Affects
F-01	User Registration	Email + password signup. Language selector (mr/hi/en). Caregiver phone. Saves profile to Firebase RTDB.	App
F-02	User Login	Email + password. Persists session via AsyncStorage. Auto-login on reopen.	App
F-03	Add Medicine	Form: name, dose, times (multiple), expiry, photo, language, active toggle. Validates all fields. Uploads photo. Saves to Firebase.	App + Hardware
F-04	Edit Medicine	Pre-filled form. Update any field. Cancels old notifications; schedules new ones.	App
F-05	Delete Medicine	Swipe-left on card or menu. Removes from Firebase. Cancels all notifications for that medicine.	App
F-06	Today Schedule Screen	FlatList sorted by time. Each card: photo thumbnail, name, dose, next-time, taken/pending status, expiry badge.	App
F-07	Push Notification	Local notification fires at each scheduled time. Shows medicine name. Works background/killed app.	App
F-08	Reminder Full-Screen	Opens on notification tap. Shows large tablet photo. Auto-speaks in selected language. Mark Taken + Snooze buttons.	App
F-09	Mark as Taken	Green button. Updates taken_today: true in Firebase. Logs to history. Cancels caregiver 30-min timer.	App + Cloud
F-10	Voice Reminder (App)	expo-speech auto-fires on ReminderScreen. Language: mr-IN / hi-IN / en-IN. Replay button on screen.	App
F-11	ESP32 Speaker Alert	Hardware plays {name}_{lang}.mp3 from SD card at scheduled RTC time. I2S → MAX98357A → 3W speaker.	Hardware
F-12	LED + Buzzer Alert	3-cycle blink + beep on ESP32 at reminder time. Backup if audio fails.	Hardware
F-13	Expiry Tracking	Red badge on card if expiry ≤ 7 days. Orange badge if ≤ 30 days. Green if safe. Shows days remaining.	App
F-14	Multilingual — 3 Langs	All UI strings in mr/hi/en. Voice speaks in selected lang. Speaker audio in selected lang.	App + Hardware
F-15	Offline ESP32	Last successful Firebase fetch saved to SPIFFS. ESP32 loads SPIFFS if WiFi unavailable. Alerts still fire.	Hardware
F-16	Multiple Time Slots	One medicine can have multiple times per day (e.g. 08:00, 14:00, 20:00). Each slot gets its own notification.	App + Hardware

ID	Feature Name	Description	Affects
F-17	Snooze Reminder	Snooze 15 or 30 min from ReminderScreen. Reschedules notification. Re-triggers ESP32 on next poll.	App

6.2 Should-Have Features (Version 1.1 — Post-Submission)

ID	Feature Name	Description
F-18	Caregiver WhatsApp Alert	If taken_today remains false 30 min after reminder time → Twilio API sends WhatsApp message to caregiver phone with medicine name and patient name.
F-19	Adherence History Chart	History screen shows weekly bar chart (react-native-chart-kit). % of doses taken on time per day. Overall weekly % score.
F-20	Caregiver Read-Only View	Family member logs in as "Caregiver" role. Gets read-only view of patient schedule + adherence. Cannot edit anything.
F-21	Doctor Report Export	Export last 30 days of adherence as PDF (expo-print). Shows medicine-by-medicine taken/missed table. Shareable via WhatsApp.
F-22	Pill Count Tracker	User sets starting pill count when adding medicine. App decrements count each time "Mark Taken" is pressed. Alert when count falls below threshold.
F-23	Medicine Autocomplete	Local JSON database of 200 common Indian medicine names. Autocomplete in name field. Auto-fills common dose.
F-24	Food Condition Tags	User tags medicine as "Before food" / "After food" / "With water". Shown prominently on ReminderScreen.
F-25	Home Screen Widget	Android/iOS home screen widget showing next medicine time. Requires expo-widgets or native module.

6.3 Nice-to-Have Features (Version 2.0 — Future)

- AI medicine scanner — point camera at tablet strip; Google ML Kit OCR reads medicine name automatically
- RFID pill compartment — smart box opens correct slot at reminder time
- WhatsApp bot reply — caregiver can reply "taken" to alert; app marks dose as taken automatically
- Health app integration — sync adherence data to Google Fit / Apple Health
- Multi-patient household — one app account manages multiple family member profiles
- Doctor appointment integration — add upcoming appointments alongside medicines

7. React Native App — Complete Step-by-Step Build

7.1 Environment Setup (Day 1)

Complete these steps before writing any app code. Spend 2–3 hours here to avoid setup issues later.

Step 1 — Install Node.js and Expo CLI

16. Download Node.js LTS (v20.x) from nodejs.org — click "LTS" button, install with defaults
17. Open Terminal (Windows: Command Prompt or PowerShell; Mac: Terminal)
18. Verify installation: node --version (should show v20.x) and npm --version (should show 10.x)
19. Install Expo CLI globally: npm install -g expo-cli
20. Verify: expo --version (should show 6.x or later)

Step 2 — Create Project and Install All Packages

Run these commands exactly in order. Each line is a separate command. Wait for each to finish before running the next.

Command	What It Does
npx create-expo-app MediMate --template blank	Creates new project in /MediMate folder with minimal template
cd MediMate	Enter the project folder
npx expo install firebase	Firebase SDK (RTDB, Storage, Auth)
npx expo install @react-navigation/native @react-navigation/stack	Navigation library
npx expo install react-native-screens react-native-safe-area-context	Required by navigation
npx expo install expo-notifications expo-device	Push notifications
npx expo install expo-speech	Text-to-speech (Marathi/Hindi/English)
npx expo install expo-image-picker	Camera and photo gallery access
npx expo install expo-av	Audio playback
npx expo install expo-print	PDF export for doctor reports
npx expo install @react-native-async-storage/async-storage	Local data persistence
npx expo install react-native-localize	Device language detection
npm install i18n-js	Translation string management
npm install react-native-chart-kit	Adherence history charts
npx expo install react-native-svg	Required by chart-kit
npm install date-fns	Date utilities

Command	What It Does
npx expo start	Start development server (do this to test)

Step 3 — Install Expo Go on Your Phone

21. Open Google Play Store (Android) or App Store (iOS)
22. Search "Expo Go" and install the official Expo app (blue logo)
23. Run npx expo start in your terminal — a QR code appears
24. Open Expo Go → Scan QR code → your app runs live on your phone
25. Any change you save in VS Code updates the app immediately (hot reload)

Step 4 — Set Up VS Code (Recommended Editor)

26. Download VS Code from code.visualstudio.com
27. Install extensions: ES7+ React/Redux Snippets, Prettier, ESLint, React Native Tools
28. Open your MediMate folder: File → Open Folder → select MediMate
29. Create .prettierrc file in root: { "semi": true, "singleQuote": true, "tabWidth": 2 }

7.2 Project Folder Structure

Create this exact folder and file structure inside your MediMate project. Delete App.js content and replace with the navigation setup below.

File / Folder Path	Purpose	Build in Week
App.js	Root file — NavigationContainer + Stack.Navigator setup	Week 6
src/	All source code lives here	—
src/screens/SplashScreen.js	Loading screen: checks auth state, redirects to Home or Auth	Week 6
src/screens/AuthScreen.js	Login + Register combined; language selector on register	Week 6
src/screens/HomeScreen.js	Today's medicine schedule with FlatList and medicine cards	Week 8
src/screens/AddMedicineScreen.js	Form to add new medicine: all fields + photo + time picker	Week 7
src/screens/EditMedicineScreen.js	Same form pre-filled with existing medicine data	Week 8
src/screens/ReminderScreen.js	Full-screen reminder: tablet photo + auto-voice + action btns	Week 9
src/screens/HistoryScreen.js	Weekly adherence chart + medicine-by-medicine breakdown	Week 11
src/screens/SettingsScreen.js	Language, caregiver phone, alert delay, profile, logout	Week 11

File / Folder Path	Purpose	Build in Week
src/screens/CaregiverScreen.js	Read-only view for caregiver login	Week 11
src/components/MedicineCard.js	Reusable card: photo, name, dose, time, badges, mark-taken btn	Week 8
src/components/ExpiryBadge.js	Color-coded badge: days until expiry, red/orange/green	Week 8
src/components/TimePicker.js	Time slot selector with add/remove buttons	Week 7
src/components/LanguagePicker.js	Dropdown: Marathi / Hindi / English	Week 7
src/firebase/config.js	Firebase app initialization with your project credentials	Week 6
src/firebase/medicines.js	All RTDB read/write functions for medicines	Week 7
src/firebase/auth.js	Login, register, logout, getCurrentUser functions	Week 6
src/firebase/history.js	Log taken/missed events; read weekly history	Week 11
src/i18n/index.js	i18n setup: loads correct translation based on user language	Week 9
src/i18n/en.js	All English UI strings	Week 9
src/i18n/hi.js	All Hindi UI strings	Week 9
src/i18n/mr.js	All Marathi UI strings	Week 9
src/utils/notifications.js	Schedule, cancel, handle notification interactions	Week 9
src/utils/speech.js	speakReminder(name, language) utility	Week 9
src/utils/caregiver.js	Start 30-min timer; call Twilio if not taken	Week 11
src/utils/storage.js	AsyncStorage helpers: get, set, remove, clear	Week 6
src/utils/dateHelpers.js	formatTime, getDaysUntilExpiry, isToday helpers	Week 7
src/constants/colors.js	App color palette constants (primary, accent, semantic)	Week 6
src/constants/languages.js	Language code mappings and TTS locale codes	Week 9
assets/medicines.json	Local JSON: 200+ common Indian medicine names for autocomplete	Week 10

7.3 Firebase Configuration (src/firebase/config.js)

Get these values from Firebase Console → Project Settings → Your Apps → Register Web App. Never commit this file to GitHub — add it to .gitignore.

Step	Action
1	Go to console.firebase.google.com → click Add project → name it "medimate-2026"
2	Disable Google Analytics (not needed) → Create Project → wait ~30 seconds
3	Click the Web icon (</>) → Register app as "MediMate-RN" → copy the firebaseConfig object
4	In Firebase Console: Authentication → Get started → Email/Password → Enable → Save
5	In Firebase Console: Realtime Database → Create database → Start in test mode → Choose your region (asia-south1 for India)
6	In Firebase Console: Storage → Get started → Start in test mode → Confirm
7	In Firebase Console: Project Settings → Service accounts → Generate new private key (for Cloud Functions later)
8	Paste your firebaseConfig into src/firebase/config.js (code pattern below)

src/firebase/config.js — paste your actual values from Firebase Console

Variable	Where to Find It in Firebase Console
apiKey	Project Settings → Your Apps → Web API Key
authDomain	Automatically: {projectId}.firebaseapp.com
databaseURL	Realtime Database → Copy the URL shown at top of database
projectId	Project Settings → General → Project ID
storageBucket	Storage → Copy the bucket URL (ends in .appspot.com)
messagingSenderId	Project Settings → Cloud Messaging → Sender ID
appId	Project Settings → Your Apps → App ID

7.4 Screen-by-Screen Complete Specification

Screen 1 — SplashScreen (src/screens/SplashScreen.js)

First screen shown every time app opens. Checks if user is logged in. Redirects automatically.

Element	Details
Background	App primary color (navy). Centered MediMate logo and name.
Loading indicator	ActivityIndicator spinner, white color
Auth check logic	onAuthStateChanged(auth, user => user ? navigate("Home") : navigate("Auth"))
AsyncStorage check	Also load saved language preference into global i18n config
Timeout	If auth check takes > 5 seconds, navigate to Auth with error message

Screen 2 — AuthScreen (src/screens/AuthScreen.js)

Element	Type	Validation Rule	Details
Mode toggle	Pressable tabs	—	"Login" and "Register" tabs. Active tab highlighted with underline.
Email input	TextInput	Required; valid email format	keyboardType="email-address"; autoCapitalize="none"
Password input	TextInput	Required; min 8 characters	secureTextEntry with show/hide eye icon toggle
Name input	TextInput	Required on Register	autoCapitalize="words"; only shown in Register mode
Language picker	Modal picker	Required on Register	Options: Marathi / Hindi / English. Saved to RTDB and AsyncStorage.
Caregiver phone	TextInput	Optional on Register	keyboardType="phone-pad"; include country code (+91 India)
Submit button	Pressable	—	Shows ActivityIndicator while Firebase call is in progress
Error message	Text	—	Red text below button. Firebase auth error codes mapped to readable messages.
Firebase Register	Code action	—	createUserWithEmailAndPassword → set user profile in RTDB → navigate Home
Firebase Login	Code action	—	signInWithEmailAndPassword → load user profile → navigate Home
Forgot Password	Text link	—	sendPasswordResetEmail. Shows success message.

Screen 3 — HomeScreen (src/screens/HomeScreen.js)

Section	Element	Details
Header	Greeting text	"Good morning, [Name]" (changes with time of day). Localized.
Header	Date	Today's date formatted: "Monday, 24 March 2026". Localized.
Summary	3 metric cards in a row	Total Medicines Today | Taken (green) | Pending (orange). Tapping Pending scrolls to first pending card.
Expiry alert	Orange banner	Shows if any medicine expires within 7 days. Tap to see which one.
Medicine list	FlatList	Sorted by next scheduled time. Each item = MedicineCard component.
MedicineCard	Tablet photo	Left side: 60x60px rounded image from Firebase Storage URL. Placeholder if no photo.
MedicineCard	Name + dose	Bold name (18pt). Dose below in gray (14pt).
MedicineCard	Next time	"Next: 14:00" or "Next: Tomorrow 08:00". Calculated from times array.
MedicineCard	Expiry badge	ExpiryBadge component (red/orange/green pill).
MedicineCard	Mark Taken button	Green button. Disabled with checkmark if already taken today.
MedicineCard	Swipe left	Reveals Edit and Delete action buttons.
Empty state	Illustration	Simple graphic + "No medicines added yet. Tap + to add your first medicine."
FAB	Floating + button	Bottom-right. Navy background. Navigates to AddMedicineScreen.
Bottom tabs	Tab navigation	Home | History | Settings. Active tab highlighted.

Screen 4 — AddMedicineScreen (src/screens/AddMedicineScreen.js)

Field	Input Type	Validation	Behavior
Medicine Name	TextInput with autocomplete	Required; min 2 chars	Shows suggestion list from medicines.json as user types. Tap suggestion to fill.
Dose / Strength	TextInput	Optional; e.g. "500mg"	Keyboard: default. Placeholder: "e.g. 500mg, 10ml"
Reminder Times	Custom TimePicker component	At least 1 time required	Shows list of time slots. "Add time" button opens time wheel picker. Trash icon removes slot.
Expiry Date	DateTimePicker (expo)	Must be today or future date	Shows days remaining preview after selection: "Expires in 234 days"
Tablet Photo	ImagePicker buttons	Recommended but not required	Two buttons: "Take Photo" (camera) and "Choose from Gallery". Preview shown after

Field	Input Type	Validation	Behavior
			selection. Compress to 70% quality before upload.
Preferred Language	LanguagePicker dropdown	Required; inherits from settings	Marathi / Hindi / English. Determines which audio file ESP32 plays and voice language in app.
Food Condition	Segmented control / radio	Optional	Before food / After food / With water / No restriction
Dose Instructions	TextInput multiline	Optional	Free text: "Take with warm water" etc. Shown on ReminderScreen.
Active toggle	Switch (on by default)	—	When off: medicine stays in DB but no reminders fire. Useful for temporary pause.
Save button	Primary action button	—	Validates → shows loading → uploads photo → writes to Firebase → schedules notifications → navigates back to Home

Screen 5 — ReminderScreen (src/screens/ReminderScreen.js)

This is the most important screen. It opens when user taps the push notification. Design for maximum clarity — the user just woke up or is busy and needs to act immediately.

Element	Design Details
Background	Solid teal (#0F6E56) when first opened. Switches to green (#1A5C2A) after marking taken.
Tablet photo	Full-width image (aspect 1:1). Rounded corners (16px). White border. Fetched from Firebase Storage URL passed via navigation params.
Medicine name	White, 28pt bold, centered. Localized.
Dose + time	White, 16pt. "500mg — Scheduled at 08:00". Localized.
Food instruction	Amber pill badge: "Take with food" / "Take before food". High visibility.
Auto-speak	useEffect fires expo-speech 1 second after screen mounts. Speaks: "[Medicine name] lene ka samay ho gaya" in selected language.
Replay button	Small speaker icon top-right. Re-triggers speech on tap.
Mark as Taken	Full-width green button (52px height). White text "Mark as Taken" / "Ghetle" / "Li Gayi". Fires: update Firebase → log history → cancel caregiver timer → navigate Home.
Snooze 15 min	Secondary button (outlined). Fires: scheduleNotification 15 min from now → navigate Home.
Snooze 30 min	Secondary button (outlined). Same as above but 30 min.

Element	Design Details
Missed button	Small text link "I missed this dose". Fires: log as missed → start caregiver alert timer → navigate Home.

Screen 6 — HistoryScreen (src/screens/HistoryScreen.js)

Element	Details
Week selector	Left/right arrows. Shows "Week of March 17 – March 23". Swipe gesture also works.
Adherence score	Large percentage number (e.g. "87%"). Label: "Doses taken this week". Color: green >80%, amber 50-80%, red <50%.
Bar chart	react-native-chart-kit BarChart. X-axis: Mon/Tue/Wed/Thu/Fri/Sat/Sun. Y-axis: 0-100%. Bar color matches adherence color.
Medicine breakdown	List: each medicine name + (taken/total) this week. E.g. "Paracetamol: 5/6 taken".
Export button	"Share with Doctor" button. Calls expo-print → generates PDF → opens share sheet.
Data source	Reads /history/{uid}/{dates} from Firebase RTDB. Falls back to today's taken_today flags for current day.

Screen 7 — SettingsScreen (src/screens/SettingsScreen.js)

Setting	Type	Details
Profile Name	TextInput + Save button	Updates /users/{uid}/name in Firebase
App Language	Picker/dropdown	Marathi / Hindi / English. Updates AsyncStorage + i18n globally. Also updates RTDB for ESP32 language selection.
Caregiver Name	TextInput + Save button	Name of family member receiving alerts
Caregiver Phone	TextInput + Save button	WhatsApp-enabled number with country code
Alert Delay	Segmented: 15/30/60 min	How long after missed dose to send caregiver alert
ESP32 Hardware Mode	Switch	When ON: app shows ESP32 sync status. When OFF: app-only mode.
Test Hardware Alert	Button	Writes a test record to Firebase → ESP32 should announce "Test" within 60s
Notification Permissions	Status display + Fix button	Shows if notifications are enabled. "Fix" opens device settings.
Account Email	Read-only text	Logged-in email. Cannot change here.
Change Password	Button → modal	Calls sendPasswordResetEmail → shows "Check your email" confirmation

Setting	Type	Details
Sign Out	Red button	signOut() → clear AsyncStorage → navigate to Auth
App Version	Read-only text	"MediMate v1.0.0 — Build 2026.03"

7.5 Key Code Patterns

Firebase config.js

Config Field	Example Value (replace with yours)
apiKey	"AIzaSyABCDEFGHIJKLMNOPQRSTUVWXYZ123456"
authDomain	"medimate-2026.firebaseapp.com"
databaseURL	"https://medimate-2026-default-rtdb.asia-southeast1.firebasedatabase.app"
projectId	"medimate-2026"
storageBucket	"medimate-2026.appspot.com"
messagingSenderId	"123456789012"
appId	"1:123456789012:web:abcdef1234567890abcdef"

Notification Scheduling Logic (src/utils/notifications.js)

Function	Parameters	What It Does
requestPermissions()	none	Calls Notifications.requestPermissionsAsync(). Must be called once on app load. Returns granted/denied.
scheduleMedicineAlerts(med)	medicine object	Loops through med.times array. For each time: calls scheduleNotificationAsync with daily repeating trigger at that hour:minute.
cancelMedicineAlerts(medId)	medicine ID string	Gets all scheduled notifications. Cancels any whose identifier starts with medId prefix.
handleNotificationResponse()	notification response	Called when user taps notification. Extracts medicineId + language from data payload. Navigates to ReminderScreen with params.
rescheduleSnooze(med, minutes)	medicine, int	Schedules one-time notification at now + minutes. Does NOT cancel the daily repeating notification.

Speech Utility (src/utils/speech.js)

Language	expo-speech locale	Reminder text template
Marathi	mr-IN	{medicineName} gheNyachi vel zali. Aapli aushadh ghya.
Hindi	hi-IN	{medicineName} lene ka samay ho gaya. Apni dawai lijiye.
English	en-IN	Time to take {medicineName}. Please take your medicine now.

Medicine RTDB Operations (src/firebase/medicines.js)

Operation	Firebase SDK Call	Path
Add medicine	push(ref(db, "schedules/" + uid), medicineData)	/schedules/{uid}/{autoId}
Update medicine	update(ref(db, "schedules/" + uid + "/" + medId), changes)	/schedules/{uid}/{medId}
Delete medicine	remove(ref(db, "schedules/" + uid + "/" + medId))	/schedules/{uid}/{medId}
Mark as taken	update(..., { taken_today: true, taken_at: timestamp })	/schedules/{uid}/{medId}
Listen live	onValue(ref(db, "schedules/" + uid), callback)	/schedules/{uid}
Upload photo	uploadBytes(storageRef, blob) → getDownloadURL()	/tablets/{uid}/{medId}.jpg

App.js — Navigation Setup

Screen Name	Route Name	When Shown	Header
SplashScreen	"Splash"	App launch; checks auth	Hidden
AuthScreen	"Auth"	Not logged in	Hidden
HomeScreen	"Home"	Main screen; logged in	Custom with greeting
AddMedicineScreen	"AddMed"	Tapped + FAB	"Add Medicine" with back arrow
EditMedicineScreen	"EditMed"	Tapped Edit on a card	"Edit Medicine" with back arrow
ReminderScreen	"Reminder"	Notification tapped	Hidden (full-screen experience)
HistoryScreen	"History"	Bottom tab: History	"Adherence History"
SettingsScreen	"Settings"	Bottom tab: Settings	"Settings"

8. ESP32 Hardware — Complete Step-by-Step Build

8.1 Bill of Materials with Exact Sources

Component	Exact Model	Qty	Price (Rs)	Where to Buy
Microcontroller	ESP32 DevKit C — 30 pin (NOT 38 pin)	1	350	Robu.in / Quartzcomponents.com / Amazon India
I2S Amplifier	MAX98357A breakout board (Adafruit clone)	1	120	Robu.in — search "MAX98357A"
Speaker	3W 8 Ohm — round 40mm or 50mm	1	80	Robu.in / any electronics shop
RTC Module	DS3231 — with coin cell battery slot	1	70	Amazon / Robu.in — DS3231 comes with CR2032 battery
SD Card Module	Micro SD card SPI module (6-pin)	1	60	Robu.in — search "micro SD module SPI"
MicroSD Card	4GB or 8GB Class 10 MicroSD	1	80	Any mobile accessory shop; also Amazon
Active Buzzer	5V active buzzer (NOT passive)	1	20	Any electronics shop; Robu.in
LED	5mm Red LED (high brightness)	1	5	Any electronics shop
Resistor	220 Ohm (red-red-brown) x5 pack	1	10	Any electronics shop
Jumper Wires	Male-Female x40 + Male-Male x40	1	80	Robu.in / Amazon — "jumper wire 40 piece"
Breadboard	Full-size 830-point breadboard	1	80	Robu.in / Amazon
USB Cable	Micro-USB (for ESP32 DevKit C programming)	1	40	Any mobile shop
CR2032 Battery	For DS3231 RTC (usually comes with module)	1	15	Any pharmacy or supermarket
	TOTAL ESTIMATED COST		~Rs 1010	

8.2 Complete Wiring Table

Wire exactly as specified below. Wiring errors are the #1 cause of hardware failures. Double-check every connection before powering on. Use 3.3V for all logic signals unless the table says 5V.

DS3231 RTC Module → ESP32

DS3231 Pin	ESP32 Pin	Wire Color (suggestion)	Notes
VCC	3.3V	Red	Do NOT connect to 5V — will damage DS3231
GND	GND	Black	Any GND pin on ESP32
SDA	GPIO 21	Blue	I2C Data — must be GPIO21 for default Wire library
SCL	GPIO 22	Yellow	I2C Clock — must be GPIO22 for default Wire library
32K	Not connected	—	Leave floating
SQW	Not connected	—	Leave floating

MAX98357A I2S Amplifier → ESP32

MAX98357A Pin	ESP32 Pin	Wire Color (suggestion)	Notes
VIN	5V (VIN pin on ESP32)	Red	MUST be 5V, not 3.3V. Use VIN pin on ESP32 which is connected to USB power.
GND	GND	Black	Any GND pin
SD	Not connected OR 3.3V	Orange	Shutdown pin — leave floating for always-on, or pull to 3.3V
GAIN	Not connected	—	Leave floating for 9dB default gain
DIN	GPIO 25	Green	I2S Data In — carries audio data
BCLK	GPIO 26	Blue	I2S Bit Clock
LRC	GPIO 27	Purple	I2S Word Select / Left-Right Clock

SD Card Module → ESP32 (SPI)

SD Module Pin	ESP32 Pin	Notes
VCC	3.3V	Some modules have onboard regulator — check your module; most work on 3.3V
GND	GND	Any GND
MOSI	GPIO 23	SPI Master Out Slave In
MISO	GPIO 19	SPI Master In Slave Out
SCK	GPIO 18	SPI Clock
CS	GPIO 5	SPI Chip Select — used to address the SD card

LED and Buzzer → ESP32

Component	Pin	ESP32 Pin	Notes
LED Anode (+)	Long leg	GPIO 2 (via 220 Ohm resistor)	Insert resistor in series between GPIO and LED anode
LED Cathode (-)	Short leg	GND	Direct connection to any GND
Active Buzzer (+)	Longer leg (marked +)	GPIO 4	ACTIVE buzzer only — it has internal oscillator
Active Buzzer (-)	Shorter leg	GND	Direct connection to GND

8.3 Arduino IDE Complete Setup

30. Download Arduino IDE 2.x from arduino.cc/en/software — choose your OS (Windows/Mac/Linux)
31. Install Arduino IDE. Open it. Go to: File → Preferences → Additional Boards Manager URLs
32. Paste this URL exactly: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json → Click OK
33. Go to: Tools → Board → Boards Manager → search "esp32" → install "esp32 by Espressif Systems" version 2.0.x (NOT 3.x yet — compatibility issues)
34. Go to: Tools → Board → ESP32 Arduino → ESP32 Dev Module
35. Go to: Tools → Port → select your ESP32 port (Windows: COMx, Mac: /dev/cu.usbserial-xxx)
36. If port not visible: your ESP32 needs CP2102 or CH340 USB-serial driver. Download from Silicon Labs (CP2102) or WCH (CH340) based on your ESP32 chip markings
37. Test connection: File → New → paste "void setup(){Serial.begin(115200);} void loop(){Serial.println("ESP32 OK"); delay(1000);}" → Upload → Open Serial Monitor → should print "ESP32 OK"

Library Installation (Tools → Manage Libraries)

Library Name	Author	Version	Purpose
RTClib	Adafruit	Latest	DS3231 RTC — read/write time
ArduinoJson	Benoit Blanchon	6.x	Parse Firebase REST JSON response
ESP32-audioI2S	schreibfaul1	Latest	Play MP3 from SD card via I2S
SD	Arduino	Built-in	Read files from microSD card
HTTPClient	Built-in ESP32	—	HTTP GET/POST for Firebase REST API
WiFi	Built-in ESP32	—	WiFi connectivity

Library Name	Author	Version	Purpose
SPIFFS	Built-in ESP32	—	Store offline schedule in ESP32 flash
Wire	Built-in Arduino	—	I2C for DS3231

8.4 Audio File Generation — Run on PC Before Programming ESP32

These MP3 files are generated on your PC using Python, then copied to the SD card. Do this BEFORE programming the ESP32. You need one file per medicine per language.

38. Install Python 3.10+ from python.org if not installed. Verify: python --version
39. Install gTTS: open terminal → pip install gtts
40. Create a file called generate_audio.py on your desktop with this logic:

Variable	Value / Pattern
medicines list	List all your medicines exactly as named in the app: ["Paracetamol", "Metformin", "Aspirin", "Amlodipine"]
languages	Dictionary: {"mr": "mr", "hi": "hi", "en": "en"}
text template MR	"[name] gheNyachi vel zali. Aapli aushadh ghya."
text template HI	"[name] lene ka waqt ho gaya. Apni dawai lijiye."
text template EN	"Time to take [name]. Please take your medicine."
output filename	"{medicineName}_{languageCode}.mp3" — e.g. Paracetamol_mr.mp3
save location	Create a folder called "audio" on your desktop. All files go here.

41. Run: python generate_audio.py — it creates all MP3 files in the audio folder
42. Also create: test_mr.mp3, test_hi.mp3, test_en.mp3 (for hardware test button in Settings)
43. Format your MicroSD card as FAT32 (Windows: right-click → Format → FAT32)
44. Copy ALL .mp3 files to the ROOT of the SD card (not inside any folder)
45. Insert SD card into the SD card module

8.5 ESP32 Firmware Architecture & Logic

Function / Block	When Called	What It Does
setup()	Once on power-on	Init Serial, WiFi, I2C (Wire), RTC, SD card, I2S audio, SPIFFS. Sync time from NTP. First Firebase fetch.
connectWiFi()	In setup()	WiFi.begin(ssid, pass). Waits up to 15 seconds. If fails, loads schedule from SPIFFS.
syncTimeFromNTP()	In setup() after WiFi	configTime(19800, 0, "pool.ntp.org") — IST offset = +5:30 = 19800 seconds. Syncs DS3231 to internet time.

Function / Block	When Called	What It Does
loop()	Continuously	Runs main logic. Uses millis() to poll Firebase every 60s without blocking audio.
fetchSchedule()	Every 60 seconds	HTTPClient GET to Firebase REST URL. Returns JSON string. Calls saveToSPIFFS on success.
parseAndCheck(json)	After fetchSchedule()	ArduinoJson parses JSON. Loops medicines. Gets DS3231 time. If hour:minute matches any medicine time → calls triggerAlert().
triggerAlert(n, l)	On time match	Param n=medicine name, l=language code. Blinks LED 3x. Beeps buzzer 3x. Calls playAudio(n + "_" + l + ".mp3").
playAudio(filename)	Inside triggerAlert()	audio.connecttoSD(filename.c_str()). Audio library handles I2S streaming to MAX98357A.
audio.loop()	Every loop() iteration	ESP32-audioI2S requires this call to keep streaming audio. Must NOT be blocked by delays.
saveToSPIFFS(json)	After successful fetch	Opens /schedule.json in SPIFFS with write mode. Writes full JSON string. Closes.
loadFromSPIFFS()	When WiFi fails	Opens /schedule.json in SPIFFS. Reads JSON string. Returns it for parseAndCheck().
resetAtMidnight()	When hour==0 && min==0	PATCH request to Firebase to set taken_today: false for all medicines. Runs only once per day (use a flag).
checkAlreadyTriggered()	Before triggerAlert()	Checks a local Set/map of "triggered today" entries. Prevents the same medicine firing every minute all day.

8.6 ESP32 Firmware Key Variables

Variable	Type	Value / Purpose
ssid	const char*	Your home WiFi network name (2.4GHz ONLY — ESP32 does not support 5GHz)
password	const char*	Your WiFi password
firebaseURL	const char*	"https://YOUR-PROJECT.firebaseio.com/schedules/YOUR-UID.json" — from Firebase Console
userUID	const char*	Your Firebase user UID — visible in Firebase Console → Auth → Users table
LED_PIN	const int	2 — GPIO connected to LED anode
BUZZER_PIN	const int	4 — GPIO connected to buzzer
I2S_DOUT	const int	25 — MAX98357A DIN
I2S_BCLK	const int	26 — MAX98357A BCLK

Variable	Type	Value / Purpose
I2S_LRC	const int	27 — MAX98357A LRC
SD_CS	const int	5 — SD card chip select
FETCH_INTERVAL	const long	60000 — fetch Firebase every 60,000 ms = 60 seconds
lastFetchTime	unsigned long	millis() timestamp of last Firebase fetch
triggeredToday	std::set<String>	Stores "{medId}_{time}" strings already triggered today — prevents repeat firing

9. Firebase Backend — Complete Setup Guide

9.1 Security Rules — Realtime Database

Paste these rules in Firebase Console → Realtime Database → Rules tab → Publish. These rules allow ESP32 to read schedules while keeping user data private.

Rule Path	Permission	Reason
/users/$uid	Read + Write: $uid === auth.uid only	Each user can only access their own profile data
/schedules/$uid	Write: $uid === auth.uid only	Only the account owner can add/edit/delete their medicines
/schedules/$uid	Read: true (public read)	ESP32 reads without authentication — acceptable for home network. Change to auth.uid for production.
/history/$uid	Read + Write: $uid === auth.uid only	Adherence history is private to the user
/caregiver_requests/$uid	Write: $uid === auth.uid only	Only user can create caregiver access requests
Firebase Storage: /tablets/$uid/	Read + Write: request.auth.uid == $uid	Photos only accessible to the account owner

9.2 Firebase Cloud Functions Setup

46. Install Node.js if not installed (required for Functions). Verify: node --version
47. Install Firebase CLI: npm install -g firebase-tools
48. Login: firebase login (opens browser → sign in with your Google account)
49. In your project folder (NOT MediMate): firebase init functions → choose your project → select JavaScript → NO to ESLint → YES to install dependencies
50. This creates a functions/ folder. Open functions/index.js

Cloud Function 1 — Midnight Schedule Reset

Property	Value
Trigger type	Scheduled (Cloud Scheduler / PubSub)
Cron schedule	"0 0 * * *" — runs at 00:00 midnight every day
Timezone	"Asia/Kolkata" (IST)
What it does	Reads all /schedules entries. Sets taken_today: false and taken_at: null for all medicines. Writes yesterday's summary to /history/{uid}/{yesterday}.
Runtime	Node.js 18
Deploy command	firebase deploy --only functions:midnightReset

Cloud Function 2 — Caregiver Alert (Optional, Twilio)

Property	Value
Trigger type	RTDB onWrite trigger on /schedules/{uid}/{medId}/taken_today
Logic	When taken_today is written to false AND current time > reminder time + alert_delay_min: call Twilio WhatsApp API
Environment vars	Set TWILIO_SID, TWILIO_TOKEN, TWILIO_FROM in Firebase Functions config (firebase functions:config:set)
Deploy command	firebase deploy --only functions:caregiverAlert

51. Deploy all functions: firebase deploy --only functions
52. Verify in Firebase Console → Functions → your functions appear with green status

9.3 Firebase Storage Rules

Rule	Purpose
allow read: if request.auth != null && request.auth.uid == userId;	Only authenticated owner can read their photos
allow write: if request.auth != null && request.auth.uid == userId && request.resource.size < 500 * 1024 && request.resource.contentType.matches("image/.*");	Only owner can write; max 500KB; must be image type

9.4 Firebase Project Checklist

Item	Status Check	Where to Verify
Project created	Project appears in Firebase Console dashboard	console.firebase.google.com
Web app registered	firebaseConfig object copied to config.js	Project Settings → Your Apps
Email auth enabled	Email/Password shows as Enabled	Authentication → Sign-in method
RTDB created	Database URL appears in Realtime Database	Realtime Database
RTDB rules published	Rules tab shows your custom rules	Realtime Database → Rules
Storage created	Storage bucket URL appears	Storage
Storage rules published	Rules tab shows your custom rules	Storage → Rules
Cloud Functions deployed	Functions show in Functions dashboard	Functions

Item	Status Check	Where to Verify
FCM enabled	Server key exists under Cloud Messaging	Project Settings → Cloud Messaging

10. Multilingual System (Marathi / Hindi / English)

10.1 Language Architecture

Layer	How Language Is Applied
App UI text	i18n-js library with translation JSON files. t("key") returns correct string based on current language.
In-app voice	expo-speech with locale: language === "mr" ? "mr-IN" : language === "hi" ? "hi-IN" : "en-IN"
Push notification	Notification body text constructed using i18n string for user's language at schedule time
ESP32 audio	Plays {medicineName}_{language}.mp3 from SD card. Language code comes from Firebase schedule record.
Language storage	User sets language on register. Stored in: AsyncStorage ("userLanguage"), RTDB (/users/{uid}/language), i18n instance

10.2 Complete Translation Keys — All Three Languages

Key	English (en)	Hindi (hi)	Marathi (mr)
app.name	MediMate	MediMate	MediMate
auth.login	Login	Login	Login
auth.register	Create Account	Account banayein	Khate banava
auth.email	Email Address	Email pata	Email patta
auth.password	Password	Password	Password
auth.name	Your Name	Aapka naam	Aapale naav
auth.language	Reminder Language	Yaad ki bhaasha	Aathavanyachi bhaasha
auth.caregiver	Caregiver Phone (optional)	Dekhbhal karne wale ka number	Seva karyakartya cha number
home.greeting_morning	Good morning	Suprabhat	Shubha praabhat
home.greeting_afternoon	Good afternoon	Namaste	Namaskaar
home.greeting_evening	Good evening	Shubh sandya	Shubh sandhya
home.total	Today's Medicines	Aaj ki dawaiyan	Aajchi aushadhe
home.taken	Taken	Li gayi	Ghetle
home.pending	Pending	Baki hai	Baaki ahe
home.empty	No medicines added yet	Koi dawai nahi jodi gayi	Koni aushadh jodli nahi
reminder.speak_mr	— (Marathi text to speak)	— (Hindi text to speak)	{name} gheNyachi vel zali. Aapli aushadh ghya.

Key	English (en)	Hindi (hi)	Marathi (mr)
reminder.speak_hi	— (English text to speak)	{name} lene ka waqt ho gaya. Apni dawai lijiye.	—
reminder.speak_en	Time to take {name}. Please take your medicine.	—	—
reminder.mark_taken	Mark as Taken	Li gayi	Ghetle
reminder.snooze_15	Snooze 15 min	15 min baad	15 min nantar
reminder.snooze_30	Snooze 30 min	30 min baad	30 min nantar
reminder.missed	I missed this dose	Mujhse yeh dose choot gayi	Maze ya dose chukle
add.title	Add Medicine	Dawai jodein	Aushadh joda
add.name	Medicine Name	Dawai ka naam	Aushadhache naav
add.dose	Dose / Strength	Khuraak	Matra
add.times	Reminder Times	Yaad dilane ka waqt	Aathavanyachi vel
add.expiry	Expiry Date	Samapti tithi	Meyadsamapti tithi
add.photo	Tablet Photo	Dawai ki tasveer	Aushadhache chitra
add.language	Reminder Language	Yaad ki bhaasha	Aathavanyachi bhaasha
add.save	Save Medicine	Dawai save karein	Aushadh save kara
settings.title	Settings	Settings	Settings
settings.language	App Language	App ki bhaasha	App chi bhaasha
settings.logout	Sign Out	Sign Out	Sign Out
expiry.days	{n} days left	{n} din bache	{n} diwas shillak
expiry.today	Expires today!	Aaj expire ho raha hai!	Aaj meyadsamapti!
expiry.expired	Expired — do not use	Expire ho gaya — use na karein	Meyadsamapt — vaparoo naka

11. Caregiver Alert System (WhatsApp via Twilio)

11.1 Alert Flow

53. User sets caregiver phone number in Settings (e.g. +919876543210 — mother's WhatsApp number)
54. Alert delay configured: 15 / 30 / 60 minutes (default: 30 minutes)
55. When a reminder fires: app starts a local timer in the background (via setTimeout or BackgroundFetch)
56. If user taps "Mark as Taken" before timer expires: timer is cancelled → no alert sent
57. If timer expires and taken_today is still false in Firebase: Twilio API is called
58. Twilio sends WhatsApp message to caregiver: "MediMate Alert: Rohit has not taken Paracetamol 500mg (scheduled at 08:00). Please check on them."
59. Message is sent in user's preferred language

11.2 Twilio Setup Steps

60. Go to twilio.com → Create free account → Verify your phone number
61. In Twilio Console: Messaging → Try WhatsApp → follow sandbox setup instructions
62. You will get a Twilio sandbox number (usually +14155238886) and a join code
63. Your caregiver must send "join [your-code]" to the Twilio sandbox number once to activate
64. Note your: Account SID, Auth Token, Twilio WhatsApp number
65. Store these in Firebase Functions config (never in app code): firebase functions:config:set twilio.sid="..." twilio.token="..." twilio.from="whatsapp:+14155238886"

11.3 WhatsApp Message Templates

Language	Alert Message (missed dose)
English	MediMate Alert: {patientName} has not taken {medicineName} ({dose}) scheduled at {time}. Please check on them.
Hindi	MediMate Alert: {patientName} ne {time} par {medicineName} ({dose}) nahi li. Kripaya unka dhyan rakhein.
Marathi	MediMate Alert: {patientName} yanni {time} la {medicineName} ({dose}) gheytle nahi. Kripaya laxa ghalava.

12. Database Schema — Complete Reference

12.1 Firebase Realtime Database Structure

Path	Field	Type	Example Value	Required?	Notes
/users/{uid}	name	String	Rohit Pawar	Yes	Display name from registration
/users/{uid}	email	String	rohit@gmail.com	Yes	Login email
/users/{uid}	language	String	mr	Yes	mr / hi / en
/users/{uid}	caregiver_name	String	Aai	No	Name of caregiver
/users/{uid}	caregiver_phone	String	+919876543210	No	Include country code
/users/{uid}	alert_delay_min	Number	30	Yes	15, 30, or 60
/users/{uid}	created_at	Number	1711180000000	Yes	Unix timestamp ms
/users/{uid}	role	String	patient	Yes	"patient" or "caregiver"
/schedules/{uid}/{medId}	name	String	Paracetamol 500mg	Yes	Full medicine name with dose
/schedules/{uid}/{medId}	dose	String	500mg	No	Dose strength
/schedules/{uid}/{medId}	times	Array	["08:00","14:00"]	Yes	Array of "HH:MM" strings
/schedules/{uid}/{medId}	expiry	String	2026-12-31	Yes	ISO date YYYY-MM-DD
/schedules/{uid}/{medId}	photo_url	String	https://firebasestorage...	No	Download URL from Storage
/schedules/{uid}/{medId}	language	String	mr	Yes	Determines audio file and voice
/schedules/{uid}/{medId}	food_condition	String	after_food	No	before_food / after_food / with_water / none

Path	Field	Type	Example Value	Required?	Notes
/schedules/{uid}/{medId}	instructions	String	Take with warm water	No	Free text instructions
/schedules/{uid}/{medId}	active	Boolean	true	Yes	false = paused, no alerts
/schedules/{uid}/{medId}	taken_today	Boolean	false	Yes	Reset to false at midnight by Cloud Function
/schedules/{uid}/{medId}	taken_at	String	"08:07"	No	Time when marked taken; null if not taken
/schedules/{uid}/{medId}	snooze_until	String	"08:30"	No	If snoozed: snooze target time
/schedules/{uid}/{medId}	pill_count	Number	28	No	Remaining pill count; decrements on taken
/schedules/{uid}/{medId}	pill_threshold	Number	7	No	Alert when pill_count falls below this
/schedules/{uid}/{medId}	created_at	Number	1711180000000	Yes	Unix timestamp ms
/history/{uid}/{date}	{medId}_taken	Boolean	true	Yes	date format: "2026-03-24"
/history/{uid}/{date}	{medId}_time	String	"08:07"	No	Time actually taken
/history/{uid}/{date}	{medId}_missed	Boolean	false	No	Marked missed explicitly

12.2 Firebase Storage Structure

Storage Path	Contents	Size Limit	Set By
tablets/{uid}/{medId}.jpg	Tablet photo uploaded by user in Add Medicine screen	500 KB	React Native app — expo-image-picker + uploadBytes()

Storage Path	Contents	Size Limit	Set By
audio/{uid}/{medicineName}_{lang}.mp3	Optional cloud-generated audio (future — not in v1.0)	200 KB	Cloud Function (future enhancement)

13. API & Data Flow Specification

13.1 ESP32 → Firebase REST API Endpoints

Method	URL Pattern	Purpose	Called When
GET	https://{project}-default-rtdb.{region}.firebasedatabase.app/schedules/{uid}.json	Fetch all medicines for user	Every 60 seconds in loop()
GET	https://{project}-default-rtdb.{region}.firebasedatabase.app/users/{uid}/language.json	Get preferred language for audio	On startup; cache locally
PATCH	https://{project}-default-rtdb.{region}.firebasedatabase.app/schedules/{uid}/{medId}.json?auth={serviceToken}	Update taken_today (if IR sensor added)	On physical pill box interaction (future)
GET	http://worldtimeapi.org/api/timezone/Asia/Kolkata	Backup NTP if configTime fails	On startup if configTime unreliable

13.2 App → Firebase SDK Operations

Action	SDK Function	Firebase Path	Triggers
Register user	createUserWithEmailAndPassword(auth, email, pass)	Firebase Auth	On Register tap
Save user profile	set(ref(db, "users/" + uid), profileData)	/users/{uid}	After registration
Login	signInWithEmailAndPassword(auth, email, pass)	Firebase Auth	On Login tap
Add medicine	push(ref(db, "schedules/" + uid), medData)	/schedules/{uid}/{autoId}	On Save in AddMedicineScreen
Upload tablet photo	uploadBytes(storageRef, blob)	Storage: tablets/{uid}/{id}.jpg	Before push() call
Get photo URL	getDownloadURL(storageRef)	Storage	After uploadBytes() completes
Listen to medicines	onValue(ref(db, "schedules/" + uid), cb)	/schedules/{uid}	HomeScreen mount; updates live
Mark as taken	update(ref(db, "schedules/" + uid + "/" + id), {...})	/schedules/{uid}/{medId}	On "Mark Taken" button tap
Edit medicine	update(ref(db, "schedules/" + uid + "/" + id), {...})	/schedules/{uid}/{medId}	On Save in EditMedicineScreen

Action	SDK Function	Firebase Path	Triggers
Delete medicine	remove(ref(db, "schedules/" + uid + "/" + id))	/schedules/{uid}/{medId}	On Delete confirmation
Update settings	update(ref(db, "users/" + uid), settingsData)	/users/{uid}	On Save in SettingsScreen
Write history	set(ref(db, "history/" + uid + "/" + date), data)	/history/{uid}/{YYYY-MM-DD}	On taken/missed action; at midnight by Function
Read history	get(ref(db, "history/" + uid))	/history/{uid}	HistoryScreen mount
Sign out	signOut(auth)	Firebase Auth	On Logout tap

14. Week-by-Week Development Roadmap (12 Weeks)

Critical rule: Build hardware FIRST (Weeks 1-5), then app (Weeks 6-9), then integration (Weeks 10-12). Do NOT start the app while hardware is unproven.

Week	Phase	Daily Tasks	End-of-Week Deliverable
1	Hardware: Procurement + Basic Test	Day 1-2: Order all components from Robu.in. Day 3: Set up Arduino IDE. Install ESP32 board package. Day 4: Wire LED + buzzer on breadboard. Upload blink sketch. Day 5: Verify LED blinks and buzzer beeps on command. Fix any wiring issues.	ESP32 powers on. LED blinks. Buzzer beeps. Arduino IDE uploads firmware successfully.
2	Hardware: RTC + Audio	Day 1-2: Wire DS3231. Install RTClib. Upload DS3231 test sketch — print time to Serial Monitor every second. Set time using RTC.adjust() if wrong. Day 3-4: Wire MAX98357A + speaker. Wire SD card module. Install ESP32-audioI2S. Day 5: Generate 3 test MP3 files with gTTS. Copy to SD card. Upload audio sketch. Speaker should speak.	DS3231 shows correct time in Serial Monitor. Speaker plays MP3 file on command.
3	Hardware: WiFi + Firebase	Day 1: Write WiFi connect sketch. Test on 2.4GHz network. Day 2: Create Firebase project. Write first REST GET to Firebase test URL. Print response to Serial. Day 3-4: Structure Firebase RTDB with sample medicine data. ESP32 fetches and prints medicine names. Day 5: Parse JSON with ArduinoJson. Print each medicine name and time.	ESP32 connects to WiFi. Fetches Firebase data. Prints medicine names to Serial Monitor.
4	Hardware: Full Alert Logic	Day 1-2: Implement parseAndCheck() function. Compare RTC time to schedule. Day 3: Add triggerAlert() — LED blink + buzzer + audio playback. Day 4: Test with medicine time set 2 minutes ahead. Verify all 3 alert outputs fire. Day 5: Add triggeredToday set to prevent repeat firing. Test stays silent after first trigger.	Full hardware alert works: set a future time in Firebase → ESP32 announces at that exact time via speaker + LED + buzzer.
5	Hardware: Offline + Polish	Day 1-2: Implement saveToSPIFFS() and loadFromSPIFFS(). Test: save schedule, disable WiFi, reboot → alerts still fire. Day 3: Implement resetAtMidnight(). Test at 23:59. Day 4: Add NTP sync on startup. Verify time is correct after first power-on without manual set. Day 5: Stress test: run	ESP32 runs 8 hours without restart. Offline mode proven. NTP sync works.

Week	Phase	Daily Tasks	End-of-Week Deliverable
		continuously for 8 hours. Fix any crashes or memory leaks.	
6	App: Setup + Auth	Day 1: Install Node.js + Expo CLI. Create project. Install all packages (see Section 7.1). Day 2: Create folder structure. Set up Firebase config.js. Day 3: Build SplashScreen with auth state check. Day 4: Build AuthScreen — login + register forms, language picker. Day 5: Test: register account, login, logout all work. Firebase shows new user.	App runs on phone via Expo Go. Registration and login work with Firebase Auth.
7	App: Add Medicine Screen	Day 1: Build TimePicker component. Day 2: Build LanguagePicker component. Build ExpiryBadge component. Day 3: Build AddMedicineScreen form — all fields, validation. Day 4: Implement photo upload to Firebase Storage. Implement push() to RTDB. Day 5: Test full save flow: fill form → take photo → save → check Firebase Console shows entry.	Add Medicine screen works completely. Data + photo appear in Firebase within 3 seconds.
8	App: Home Screen + Cards	Day 1-2: Build MedicineCard component — photo, name, dose, time, expiry badge, mark-taken button. Day 3: Build HomeScreen with FlatList. Connect to Firebase onValue listener. Sort by time. Day 4: Implement swipe-to-delete and edit navigation. Day 5: Build EditMedicineScreen (reuse Add form with pre-filled data). Test edit + delete.	HomeScreen shows live medicines from Firebase. Cards show photos. Mark Taken updates Firebase live.
9	App: Notifications + Voice	Day 1: Implement notifications.js — scheduleMedicineAlerts(), cancelMedicineAlerts(). Day 2: Add notification scheduling to Add/Edit/Delete flows. Test: add medicine → notification fires at time. Day 3: Build ReminderScreen — tablet photo, auto-speak, buttons. Day 4: Test all 3 languages on ReminderScreen. Day 5: Implement snooze logic.	Push notification fires at scheduled time. Tapping opens ReminderScreen. Voice speaks in correct language.
10	Integration: App ↔ ESP32	Day 1: Add medicine in app → verify ESP32 fetches it within 60s (watch Serial Monitor). Day 2: Set time 2 min ahead → confirm speaker + phone notification fire simultaneously. Day 3: Test Mark as Taken → Firebase updates → ESP32 should not re-alert. Day 4: Test language switch → speaker uses correct language. Day 5: Fix all integration issues found.	Complete end-to-end system works: App → Firebase → ESP32 → Speaker + App notification simultaneously.
11	Enhancement Features	Day 1-2: Build HistoryScreen with chart. Implement history writing on Mark	History chart shows weekly adherence. Caregiver

Week	Phase	Daily Tasks	End-of-Week Deliverable
		Taken. Day 3: Build SettingsScreen with all options. Implement language switch across whole app. Day 4: Implement caregiver alert system (Twilio). Test WhatsApp message delivery. Day 5: Add pill count tracking and refill alert.	receives WhatsApp alert for missed dose. Settings fully functional.
12	Polish + Demo Prep	Day 1: Fix all remaining bugs found in Weeks 10-11. Day 2: Build physical enclosure for ESP32 hardware (3D print or project box). Day 3: Record demo video (3-minute walkthrough). Day 4: Update PPT with system diagram, hardware photos, app screenshots, demo video link. Day 5: Practice live demo script. Test on fresh phone. Prepare backup ESP32.	Demo-ready system. Professional enclosure. Updated PPT. 5+ demo rehearsals completed.

15. Testing Plan — Complete Checklist

15.1 App Functional Tests

Test ID	Test Case	Steps	Expected Result	Pass/Fail
T-01	User Registration	1. Open app. 2. Switch to Register. 3. Fill all fields. 4. Tap Create Account.	User created in Firebase Auth. Profile appears in RTDB. App navigates to HomeScreen.	—
T-02	User Login	1. Close app fully. 2. Reopen. 3. Enter credentials. 4. Tap Login.	App navigates to HomeScreen. User name shown in greeting.	—
T-03	Auto Login	1. Login successfully. 2. Force-close app. 3. Reopen.	App goes directly to HomeScreen without showing AuthScreen.	—
T-04	Add Medicine Basic	1. Tap +. 2. Enter name "Paracetamol". 3. Set time 5 min in future. 4. Set expiry 6 months ahead. 5. Tap Save.	Medicine appears on HomeScreen. Firebase Console shows entry. Notification scheduled.	—
T-05	Add with Photo	1. Tap +. 2. Fill all fields. 3. Tap "Take Photo". 4. Photograph a tablet. 5. Save.	Photo thumbnail visible on medicine card. Photo URL exists in Firebase.	—
T-06	Add Multiple Times	1. Tap +. 2. Add 3 time slots: 08:00, 14:00, 20:00. 3. Save.	3 separate notifications scheduled. Card shows next upcoming time.	—
T-07	Edit Medicine	1. Swipe card left. 2. Tap Edit. 3. Change name and time. 4. Save.	Card shows updated name and time. Old notification cancelled. New one scheduled.	—
T-08	Delete Medicine	1. Swipe card left. 2. Tap Delete. 3. Confirm.	Card removed from HomeScreen. Firebase entry deleted. Notifications cancelled.	—
T-09	Notification Fires	1. Add medicine with time 2 min from now. 2. Close app. 3. Wait.	Push notification appears within 60 seconds of scheduled time.	—
T-10	Notification Tap	1. Wait for notification. 2. Tap it.	ReminderScreen opens with tablet photo. Voice speaks within 2 seconds.	—
T-11	Mark as Taken	1. On ReminderScreen. 2. Tap "Mark as Taken".	taken_today: true in Firebase. History entry created. Card shows checkmark.	—
T-12	Snooze 15 min	1. On ReminderScreen. 2. Tap "Snooze 15 min".	New notification fires 15 minutes later. Original	—

Test ID	Test Case	Steps	Expected Result	Pass/Fail
			daily notification unchanged.	
T-13	Marathi voice	1. Set language to Marathi. 2. Trigger reminder.	Voice speaks in Marathi with mr-IN locale.	—
T-14	Hindi voice	1. Set language to Hindi. 2. Trigger reminder.	Voice speaks in Hindi with hi-IN locale.	—
T-15	English voice	1. Set language to English. 2. Trigger reminder.	Voice speaks in English with en-IN locale.	—
T-16	Expiry badge red	1. Add medicine with expiry = today + 3 days.	Red badge shows "3 days left" on medicine card.	—
T-17	Expiry badge orange	1. Add medicine with expiry = today + 15 days.	Orange badge shows "15 days left" on medicine card.	—
T-18	Language switch UI	1. Go to Settings. 2. Change language to Marathi. 3. Go back to Home.	All HomeScreen text is now in Marathi.	—
T-19	History chart	1. Mark several medicines taken over 3 days. 2. Open History.	Bar chart shows daily % taken. Overall weekly score shown.	—
T-20	Logout	1. Go to Settings. 2. Tap Sign Out.	Returns to AuthScreen. AsyncStorage cleared.	—

15.2 Hardware Tests

Test ID	Test Case	Expected Result
HW-01	ESP32 WiFi connects on power-on	WiFi connected LED blink pattern or "Connected" in Serial Monitor within 15 seconds
HW-02	DS3231 shows correct IST time	Serial Monitor prints current hour and minute correctly after NTP sync
HW-03	Firebase schedule fetched	Serial Monitor prints medicine names within 60 seconds of power-on
HW-04	Speaker plays audio	Correct MP3 plays when triggerAlert() is called manually
HW-05	LED blinks on alert	3 blink cycles visible when alert fires
HW-06	Buzzer beeps on alert	3 beep cycles audible when alert fires
HW-07	Alert fires at correct time	Add medicine for 2 min in future → speaker announces within 60 seconds of scheduled time

Test ID	Test Case	Expected Result
HW-08	No repeat alert	Alert fires once; does NOT fire again at next loop cycle for same medicine same time
HW-09	Offline mode	Disconnect WiFi router → reboot ESP32 → alert still fires from SPIFFS cache
HW-10	Language selection works	Change medicine language in app → ESP32 plays correct language MP3 (_mr, _hi, or _en)
HW-11	New medicine detected	Add medicine in app → ESP32 Serial Monitor shows new medicine within 60 seconds
HW-12	Runs 8 hours continuously	No restart, crash, or memory error over 8-hour continuous operation

16. Non-Functional Requirements

Category	Requirement	Target Value
Performance	App cold start time (first open)	Under 2 seconds to SplashScreen
Performance	HomeScreen load after login	Under 1 second (Firebase onValue fires fast)
Performance	App → Firebase → ESP32 sync latency	Under 60 seconds for new medicine to appear on ESP32
Performance	Notification delivery delay	Within 30 seconds of scheduled time
Performance	Photo upload time	Under 5 seconds for 500KB photo on 4G
Reliability	ESP32 continuous uptime	99%+ — must run 24 hours without restart
Reliability	App crash rate	Less than 1 crash per 100 app opens
Reliability	Notification delivery rate	Greater than 99% (local notification, not dependent on server)
Offline	ESP32 without internet	Must announce from SPIFFS cache indefinitely
Offline	App without internet	HomeScreen must load from AsyncStorage cache; cannot add medicines offline
Security	User data access	Firebase rules enforce: each user accesses only their own data
Security	Credentials	Firebase API key in config.js is safe to expose (Firebase rules protect data); service account key must NEVER go in app code
Usability	Minimum tap target size	48 x 48 density-independent pixels for all interactive elements
Usability	Minimum body text size	16pt (accessible for elderly users)
Usability	Color contrast ratio	WCAG AA: min 4.5:1 for normal text, 3:1 for large text
Compatibility	Android version	Android 8.0 (API 26) and above
Compatibility	iOS version	iOS 13.0 and above
Cost	Monthly cloud cost at 100 users	Rs 0 — Firebase Spark tier sufficient for project scale
Cost	Hardware build cost	Under Rs 1100 for complete unit

17. Risk Register & Mitigations

Risk	Probability	Impact	Mitigation Strategy
ESP32 WiFi drops / home router restarts	High	High	SPIFFS offline cache: ESP32 loads last good schedule and continues alerting without internet. Auto-reconnect in loop().
MAX98357A not producing sound	Medium	High	1. Check VIN is 5V not 3.3V. 2. Verify DIN/BCLK/LRC pin mapping matches code. 3. Check SD card is FAT32. 4. Fallback to buzzer-only alert mode always runs in parallel.
SD card file not found error	Low	High	Check file existence with SD.exists() before playAudio(). If not found: log error to Serial + use buzzer as backup. Never crash the loop.
ESP32 crashes after hours of operation	Medium	High	Test 8-hour run in Week 5. Add watchdog timer (esp_task_wdt). Use ESP.restart() on critical failures. Use static allocation where possible.
gTTS Marathi pronunciation of medicine names is wrong	Medium	Medium	Test all medicine names before SD card loading. Record custom WAV files using Audacity if gTTS pronunciation is unacceptable.
expo-speech mr-IN not supported on older Android	Medium	Medium	Test on 3 different Android versions. Fallback: if mr-IN fails, try mr, then en-IN. Always have fallback language.
Firebase free tier limits exceeded	Low	Low	Monitor usage in Firebase Console. At 100 users with photos: Storage ~20MB (free limit 5GB). Far from limits.
Twilio trial balance runs out before demo	Low	Medium	Check balance before demo day. Add balance if needed. Test Twilio 3 days before demo.
Push notifications not working on iOS	Medium	High	iOS requires APNs setup for production. For demo: use Android (notifications work in Expo Go on Android easily).
Phone runs out of battery during demo	Medium	High	Keep phone at 100% charge. Have charging cable nearby. Consider keeping demo phone plugged in during presentation.
Expo Go requires internet for some features	Low	Low	For final demo: build a standalone APK using eas build and install directly on Android phone. Not dependent on Expo Go.
Team member hardware accidentally damages ESP32	Low	High	Order 2 ESP32s from the start. Keep backup pre-programmed. Never power ESP32 from bench power supply over 5V.

18. Demo Day Script & Checklist

18.1 Pre-Demo Preparation (Day Before)

Item	Action	Responsible
ESP32 firmware	Upload latest firmware. Test speaker. Confirm WiFi connects on demo venue network.	Hardware team
SD card audio	Verify all medicine MP3 files are present. Play each one manually.	Hardware team
App on demo phone	Install latest APK or run via Expo Go. Verify login works. Clear old test medicines.	App team
Firebase data	Clear test medicines from Firebase. Fresh start for demo.	Any member
Twilio balance	Check credit. Add balance if under Rs 100.	Any member
Caregiver demo phone	Second phone to receive WhatsApp caregiver alert. Joined Twilio sandbox.	Team
Demo venue WiFi	ESP32 needs to connect. Program demo venue WiFi credentials into firmware OR use phone hotspot.	Hardware team
Backup ESP32	Second ESP32 pre-programmed and working in your bag.	Hardware team
Laptop Firebase Console	Open Firebase Console in browser. Show database updating live during demo.	Any member
PPT slides	Update with real hardware photos and app screenshots.	Any member

18.2 Live Demo Script (Aim for 4 minutes)

66. [0:00] Show HomeScreen. "This is MediMate 2.0. Currently no medicines added. Let me add one now."
67. [0:15] Tap +. Fill: Name = "Paracetamol 500mg". Dose = "500mg". Time = current time + 2 minutes. Expiry = today + 60 days. Take a photo of an actual tablet. Language = Marathi. Tap Save.
68. [0:45] Show Firebase Console on laptop. "Notice — the data appeared in the cloud database instantly. Name, time, expiry, photo URL — all there."
69. [0:55] Show ESP32 Serial Monitor on laptop. "Our hardware is now polling Firebase. Within 60 seconds it will detect this new medicine."
70. [1:10] Wait for Serial Monitor to show the medicine name. "There — the hardware has fetched it. It now knows Paracetamol needs to be announced at [time]."
71. [1:30] "Now let's wait for the scheduled time..." [pause — show other app screens: History, Settings while waiting]
72. [2:00] Speaker fires: "Paracetamol gheNyachi vel zali..." — Phone notification arrives simultaneously. "Both the physical speaker AND the phone notification fired at the same moment."
73. [2:15] Tap notification. ReminderScreen opens with tablet photo. Voice speaks Marathi again. "The app shows the exact tablet photo and speaks in Marathi."

74. [2:30] Tap "Mark as Taken". Navigate to Home — card shows checkmark. Show Firebase Console — taken_today: true visible.
75. [2:45] Switch to Hindi in Settings. Trigger another reminder. "Now hear it in Hindi." Switch to English. Same again. "And in English."
76. [3:15] Show HistoryScreen with chart. "The system tracks adherence history. This chart is useful to show doctors."
77. [3:30] "If the user doesn't mark the dose taken within 30 minutes, our caregiver alert system sends a WhatsApp message to the family member." Show the WhatsApp on second phone.
78. [3:45] "MediMate 2.0 — zero missed doses through redundant alerts in three languages." Questions?

18.3 Questions Evaluators Typically Ask

Question	Suggested Answer
What happens if ESP32 loses WiFi?	ESP32 saves the last successful schedule to its internal flash memory (SPIFFS). Even without WiFi, it reads from this cache and continues firing alerts at the correct times.
How is this different from just using a phone alarm?	Three key differences: 1) Physical speaker works even if phone is in another room or on silent. 2) Shows tablet photo so elderly users know exactly which medicine to take. 3) Caregiver alert if dose is missed — phone alarms have none of this.
What is the total cost to build?	Hardware is under Rs 1100. Cloud backend uses Firebase free tier which supports thousands of users for free. The app itself is free to distribute on Android.
Can it support multiple patients?	The current version supports one patient per account. The architecture uses UID-scoped data in Firebase, so adding multi-patient support is a straightforward extension — just add a patients collection under each UID.
How secure is the patient data?	Firebase security rules ensure each user can only access their own data. Photos are stored with storage rules requiring authentication. The Twilio API credentials are stored as server-side environment variables, not in the app.
What if the medicine name is not recognized by gTTS?	We tested all common medicine names. For unusual names, we can record custom audio files. In the future, we plan to add phonetic spelling hints when adding a medicine.

19. Future Scope & Enhancements

19.1 Version 1.1 — Short Term (3 months after submission)

Enhancement	Description	Technical Approach
AI Tablet Scanner	Point camera at tablet strip — OCR reads medicine name automatically	Google ML Kit Text Recognition API; on-device, no server needed
Doctor PDF Report	30-day adherence report in PDF format for clinic visits	expo-print + custom HTML template; share via WhatsApp or email
Pill Count Tracker	Track remaining pills; alert when refill needed	Add pill_count field to RTDB; decrement on Mark Taken; threshold alert
Food Condition Display	Show "Before food" / "After food" prominently on ReminderScreen	Add food_condition field; show as colored badge with food icon
Notification History	Log all fired reminders with timestamps; show in-app notification history	RTDB path /notification_log/{uid}/{date}; FlatList view
Multiple Patient Profiles	One account manages medicines for whole family	Add /patients/{uid}/{patientId}; scoped schedules per patient

19.2 Version 2.0 — Medium Term (6–12 months)

- Smart RFID Pillbox: physical compartmentalized box opens correct slot at reminder time; IR sensor confirms pill physically removed
- WhatsApp Bot: caregiver can reply "taken" to alert message; Cloud Function marks dose as taken in Firebase without patient using app
- Health App Sync: adherence data exported to Google Fit (Android) and Apple Health (iOS) via their APIs
- Wearable Reminder: vibration alert on connected smartwatch via Wear OS / watchOS companion app
- Medicine Interaction Checker: local database of known drug interactions; warn user if two medicines conflict
- Doctor Portal Web App: web dashboard for doctors to view all MediMate patients' adherence in one place

19.3 Version 3.0 — Long Term (Commercial Potential)

- Hospital Ward Integration: centralized nurse dashboard showing all in-patient medication adherence across a ward
- Insurance Integration: anonymized adherence data shared with health insurers; adherent patients get premium discounts
- Pharmacy Auto-Refill: when pill count reaches minimum threshold, system automatically initiates refill order with linked pharmacy
- Clinical Trial Platform: pharmaceutical companies use MediMate to monitor trial participant adherence with cryptographic audit trail

- AI Adherence Prediction: machine learning model predicts which patients are likely to miss doses based on historical patterns; proactive alerts

20. Appendix: Common Errors & Fixes

20.1 React Native App Errors

Error Message	Cause	Fix
Cannot read property of undefined (Firebase)	Firebase not initialized before first call	Move all Firebase calls inside useEffect or after auth state confirmed. Check config.js exports are correct.
Notifications not showing on Android	Permissions not requested or notification channel missing	Call Notifications.requestPermissionsAsync() on app load. For Android 13+: add POST_NOTIFICATIONS permission in app.json.
expo-speech language not supported	mr-IN or hi-IN not installed on test device	Test on physical Android device not emulator. Add fallback: catch error and retry with en-IN.
Image upload fails (Storage)	Storage rules too restrictive or CORS issue	Check Storage rules allow write for auth.uid. Ensure photo is converted to blob correctly before uploadBytes().
Navigation: undefined is not an object	Navigating before NavigationContainer is mounted	Ensure all navigation calls are inside screen components. Use optional chaining: navigation?.navigate().
AsyncStorage deprecated warning	Using old import path	Use: import AsyncStorage from "@react-native-async-storage/async-storage" (not from react-native)
Metro bundler error: module not found	Package not installed or wrong import path	Run: npx expo install [package-name]. Restart Metro with --clear: npx expo start --clear
App crashes on Android when camera opens	Missing camera permission in app.json	Add to app.json plugins: ["expo-image-picker", {"cameraPermission": "Allow MediMate to take tablet photos"}]

20.2 ESP32 Hardware Errors

Symptom	Likely Cause	Diagnostic + Fix
ESP32 port not visible in Arduino IDE	USB driver not installed	Install CP2102 driver (Silicon Labs website) for ESP32 with CP2102 chip, or CH340 driver (WCH website) for CH340 chip. Check chip marking on ESP32 board.
Upload fails: "A fatal error occurred"	Wrong COM port or reset needed	Hold BOOT button on ESP32 during upload. Release after "Connecting..." appears. Set Upload Speed to 115200 in Tools menu.
WiFi not connecting	5GHz network or wrong credentials	ESP32 supports ONLY 2.4GHz. Check your router has 2.4GHz band. Verify ssid and password are exact (case sensitive, no trailing spaces).

Symptom	Likely Cause	Diagnostic + Fix
No audio from speaker	MAX98357A wiring or power issue	1. Verify VIN of MAX98357A goes to 5V VIN of ESP32 (not 3.3V). 2. Check DIN=GPIO25, BCLK=GPIO26, LRC=GPIO27. 3. Verify SD card has FAT32 format and MP3 files in root.
DS3231 time always wrong/resets	CR2032 battery missing or dead	Insert fresh CR2032 battery into DS3231 coin cell holder. Without it, time resets every power cycle.
SD card not initialized	SPI pins wrong or card not FAT32	Verify: MOSI=23, MISO=19, SCK=18, CS=5. Format card as FAT32 (not exFAT). Try a different SD card — some 32GB+ cards use exFAT by default.
Firebase response is 401 Unauthorized	Database rules require auth but ESP32 has none	Set RTDB rule for /schedules/$uid to ".read": true for development. This is safe on home network.
ESP32 crashes randomly after hours	Memory fragmentation or stack overflow	Reduce StaticJsonDocument size. Use F() macro for string literals. Add watchdog: esp_task_wdt_init(30, true). Print free heap in loop for diagnostics.

20.3 Firebase Setup Errors

Error	Cause	Fix
Permission denied (RTDB)	Security rules blocking read/write	Firebase Console → RTDB → Rules → temporarily set to ".read": true, ".write": true for testing. Tighten rules before production.
Firebase: No such object (Storage)	Storage not initialized or wrong path	Verify Firebase Storage is created in Console. Check storageBucket in config.js matches Storage URL in Console.
Auth: email already in use	Account exists with that email	Use password reset flow. Or use a different test email for development.
Cloud Functions deploy fails	Node.js version mismatch or syntax error	Ensure Node.js 18 is installed. Check functions/index.js for syntax errors. Run: firebase functions:log to see error details.
databaseURL not found	Wrong or missing databaseURL in config.js	In Firebase Console: Realtime Database → copy the URL exactly including https:// prefix. Format: https://PROJECT-default-rtdb.REGION.firebasedatabase.app

MediMate 2.0 — Product Requirements Document

Version 1.0 | March 2026 | Sir Visvesvaraya Institute of Technology

Rohit Vijay Pawar | Tanuja Changdev Ranade | Shruti Kailas Saskar

Guide: Dr. Rana Mahajan | Dept. of Electronics and Computer Engineering